

Instrucoes de Execucao - Fase 2

Desenvolvimento de Sistemas Back-End - Fase 2

Aluno: Andreas Berwaldt

1. Visao Geral

Esta entrega implementa a arquitetura solicitada na Fase 2:

- **ServicoGestao**: mantem clientes, planos e assinaturas.
- **ServicoFaturamento**: registra pagamentos em banco proprio e publica eventos.
- **ServicoPlanosAtivos**: responde rapidamente se uma assinatura esta ativa, usando cache em memoria.
- **api-gateway**: entrada HTTP unica para testes via Postman.
- **RabbitMQ**: message broker para eventos de pagamento.
- Dois bancos PostgreSQL: um para gestao e outro para faturamento.

2. Pre-requisitos

- Node.js 20 ou superior
- Docker Desktop
- npm
- Postman, para importar a collection da entrega

3. Portas Utilizadas

Recurso	Porta
API Gateway	3000
ServicoGestao	3001
ServicoFaturamento	3002
ServicoPlanosAtivos	3003
PostgreSQL Gestao	5433
PostgreSQL Faturamento	5434
RabbitMQ	5672
RabbitMQ Management	15672

4. Estrutura de Pastas

```
Desenvolvimento-de-Sistemas-BackEnd-Fase2/  
├─ api-gateway/  
├─ servico-gestao/  
└─ servico-faturamento/
```

```
|— servico-planos-ativos/  
|— docker-compose.yml  
|— Andreas_Berwaldt_Desenvolvimento_de_Sistemas_backend_Fase-  
2.postman_collection.json  
|— Andreas_Berwaldt_relatorio.pdf  
|— Andreas_Berwaldt_instrucoes.pdf
```

5. Subir Infraestrutura

Na raiz do repositório:

```
docker compose up -d
```

Isso inicia:

- `servico-gestao-db` em `localhost:5433`
- `servico-faturamento-db` em `localhost:5434`
- `pagamentos-rabbitmq` em `localhost:5672`

O painel do RabbitMQ fica disponível em <http://localhost:15672>.

6. Variaveis de Ambiente

Crie os arquivos `.env` a partir dos `.env.example`.

`servico-gestao/.env`

```
PORT=3001  
DATABASE_URL="postgresql://postgres:postgres@localhost:5433/servico_gestao?  
schema=public"  
RABBITMQ_URL="amqp://localhost:5672"
```

`servico-faturamento/.env`

```
PORT=3002  
DATABASE_URL="postgresql://postgres:postgres@localhost:5434/servico_faturamento?  
schema=public"  
RABBITMQ_URL="amqp://localhost:5672"
```

`servico-planos-ativos/.env`

```
PORT=3003  
GESTAO_URL="http://localhost:3001"
```

```
RABBITMQ_URL="amqp://localhost:5672"
```

api-gateway/.env

```
PORT=3000  
GESTAO_URL="http://localhost:3001"  
FATURAMENTO_URL="http://localhost:3002"  
PLANOS_ATIVOS_URL="http://localhost:3003"
```

7. Instalar Dependencias

Execute em cada pasta:

```
cd servico-gestao  
npm install  
  
cd ../servico-faturamento  
npm install  
  
cd ../servico-planos-ativos  
npm install  
  
cd ../api-gateway  
npm install
```

8. Preparar Bancos

ServicoGestao

```
cd servico-gestao  
npx prisma generate  
npx prisma migrate deploy  
npm run seed
```

O seed cria 10 clientes, 5 planos e 5 assinaturas.

ServicoFaturamento

```
cd servico-faturamento  
npx prisma generate  
npx prisma migrate deploy
```

9. Iniciar os Servicos

Abra quatro terminais:

```
cd servico-gestao  
npm run start
```

```
cd servico-faturamento  
npm run start
```

```
cd servico-planos-ativos  
npm run start
```

```
cd api-gateway  
npm run start
```

Use o gateway em <http://localhost:3000> para executar os testes manuais.

10. Endpoints pelo Gateway

Base URL: <http://localhost:3000>

Metodo	Endpoint	Descricao
GET	/gestao/clientes	Lista clientes
GET	/gestao/planos	Lista planos
POST	/gestao/assinaturas	Cria assinatura
PATCH	/gestao/planos/:idPlano	Atualiza custo mensal do plano
GET	/gestao/assinaturas/:tipo	Lista assinaturas por TODOS , ATIVOS ou CANCELADOS
GET	/gestao/assinaturascliente/:codcli	Lista assinaturas de um cliente
GET	/gestao/assinaturasplano/:codplano	Lista assinaturas de um plano
POST	/registrarpagamento	Registra pagamento
GET	/planosativos/:codass	Retorna se uma assinatura esta ativa

POST [/registrarpagamento](#)

```
{  
  "dia": 14,
```

```
"mes": 6,  
"ano": 2026,  
"codAss": 1,  
"valorPago": 99.9  
}
```

Ao registrar o pagamento, o ServicoFaturamento persiste o registro e publica evento no RabbitMQ. O ServicoGestao consome o evento e atualiza `dataUltimoPagamento`; o ServicoPlanosAtivos consome o mesmo evento e remove a assinatura do cache.

11. Regra de Assinatura Ativa

Uma assinatura esta ativa quando `dataUltimoPagamento` ocorreu ha menos de 30 dias. A fidelidade (`fimFidelidade`) continua sendo armazenada, mas nao e usada como criterio de ativacao do servico.

12. Testes Automatizados

Execute:

```
cd servico-gestao && npm test  
cd ../servico-faturamento && npm test  
cd ../servico-planos-ativos && npm test  
cd ../api-gateway && npm test
```

Resultados validados:

```
servico-gestao:      27 testes passando  
servico-faturamento: 8 testes passando  
servico-planos-ativos: 9 testes passando  
api-gateway:        5 testes passando
```

13. Build

```
cd servico-gestao && npm run build  
cd ../servico-faturamento && npm run build  
cd ../servico-planos-ativos && npm run build  
cd ../api-gateway && npm run build
```

14. Teste Integrado Validado

Com os quatro servicos rodando:

```
Invoke-RestMethod http://localhost:3000/gestao/clientes  
Invoke-RestMethod http://localhost:3000/planosativos/1
```

```
Invoke-RestMethod -Method Post -Uri http://localhost:3000/registrarpagamento -  
ContentType "application/json" -Body  
'{"dia":14,"mes":6,"ano":2026,"codAss":1,"valorPago":99.9}'  
Invoke-RestMethod http://localhost:3000/planosativos/1
```

Resultado observado:

- listagem retornou 10 clientes;
- consulta de plano ativo retornou `true`;
- pagamento foi registrado com sucesso;
- nova consulta de plano ativo continuou respondendo corretamente apos a invalidacao do cache.

15. Parar Ambiente

Para parar os servicos Node, use `Ctrl+C` nos terminais.

Para parar infraestrutura:

```
docker compose down
```

Para remover tambem os dados:

```
docker compose down -v
```